Shader-Based Genomic Homology Search via Spectral Coordinate Embeddings

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
kundai.sachikonye@bitspark.com

April 25, 2026

Abstract

We describe a homology search architecture that maps nucleotide and protein sequences to short fixed-dimensional coordinates and evaluates query-versus- database similarity as a single rendering pass of a GPU fragment shader. The embedding takes the magnitudes of the lowest-frequency coefficients of the discrete Fourier transform of a channelised representation of the sequence: one-hot for DNA, and three physicochemical indices (hydropathy, side-chain volume, net charge) for protein. Under this embedding, cosine similarity of the coordinate vectors is a Lipschitz-continuous surrogate for local alignment score that preserves the ordering of homologs over non-homologs at substitution rates up to the classical twilight zone. Because the similarity kernel reduces to a single dot product per database entry, it maps without modification onto the rendering pipeline of every commodity GPU and mobile device: each fragment shader invocation computes one query–database comparison, and the framebuffer holds the per-entry similarity. We validate the architecture on synthetic homology benchmarks spanning eight substitution rates, database sizes from 10^2 to 10^5 sequences, and both nucleotide and protein alphabets. The embedding separates within-family from between-family pairs with ROC-AUC $\geq$ 0.97 up to 20% divergence and maintains recall@5 above 0.93 relative to exhaustive Smith–Waterman ground truth on protein data. A CPU reference implementation of the shader kernel sustains 1.1 ms per query against a 10^5-sequence database, four orders of magnitude faster than full exhaustive Smith–Waterman and three orders faster than full k-mer Jaccard at the same scale. A two-stage pipeline that uses the shader kernel as a first-pass filter and full local alignment only as a re-ranker on the top-K candidates recovers 95% of true homologs at $K = 20$ without further tuning. All experiments, raw data and figures are released alongside this paper.

Contents

1 Introduction **2**

2 Method **3**
2.1 Channelised sequence representation 3
2.2 Spectral embedding 3
2.3 Fragment-shader similarity kernel 3
2.4 Hierarchical coordinate addressing 5
2.5 Two-stage pipeline 5
2.6 Complexity 6

3 Experimental setup **6**
3.1 Synthetic benchmarks 6
3.2 Baselines 6
3.3 Implementation and hardware . . 6
3.4 Reported metrics 6

4 Results **8**
4.1 Separation of homologs from non-homologs 8
4.2 Ranking agreement and recall . . 8
4.3 Scaling behaviour 8
4.4 Hierarchical prefix addressing . . 10
4.5 End-to-end pipeline 10

5 Discussion **12**
5.1 Where the method wins 12
5.2 Where it loses 12
5.3 Expected GPU throughput . . . 13
5.4 Relation to prior work 13

5.5 Applications 13
5.6 Future work 14

6 Conclusion **14**

1 Introduction

Homology search—finding sequences in a large reference database that are evolutionarily or functionally related to a given query—is the single most frequently invoked primitive in computational biology. The classical formulation, due to Needleman and Wunsch [Needleman and Wunsch, 1970] and Smith and Waterman [Smith and Waterman, 1981], solves pairwise alignment by dynamic programming in $\mathcal{O}(mn)$ time for sequences of length m and n. Running this pairwise against a database of size N yields an exhaustive cost of $\mathcal{O}(mnN)$ per query, which is prohibitive at present-day database sizes. Practical tools—`BLAST` [Altschul et al., 1990, 1997], `DIAMOND` [Buchfink et al., 2015, 2021], `MMseqs2` [Steinegger and Söding, 2017], `minimap2` [Li, 2018]—achieve their sub-exhaustive throughput by heuristic seed-and-extend strategies that still inspect $\Omega(N)$ candidate subsequences and perform extensive random memory access. A parallel line of work eliminates alignment altogether and estimates sequence distances from fixed-size sketches: MinHash and its variants [Broder, 1997, Ondov et al., 2016], HyperLogLog sketches [Flajolet et al., 2007, Baker and Langmead, 2019], and locality-sensitive hashing of k-mers [Indyk and Motwani, 1998, Andoni and Indyk, 2006]. These alignment-free methods trade exact alignment score for constant-size comparison and are now the default for dataset-scale distance estimation [Vinga and Almeida, 2003, Zielezinski et al., 2017, Pierce et al., 2019, Wood et al., 2019].

The present paper takes a step further along this trajectory. We observe that:

1. Any alignment-free sketch that compares two sequences via a single fixed-dimension dot product or distance is arithmetically identical to the operation a GPU fragment shader performs, once per pixel, during rendering.

2. The rendering pipeline—not compute shaders, not CUDA, but the ubiquitous vertex/fragment path exposed by every OpenGL ES, WebGL, and Metal implementation—therefore implements a massively parallel distance query for free, on hardware that is already present in every smartphone, laptop and workstation.

3. Once the database is reduced to a coordinate matrix, the entire system has the working-set properties of a texture (bounded, cache-coherent, streamable) rather than those of a random-access FM-index.

What remains is to choose an embedding that preserves enough biological similarity for first-stage retrieval while being small enough to fit in a shader uniform. We adopt a discrete Fourier embedding of a channelised representation of the sequence, motivated by prior genomic signal-processing work [Anastassiou, 2001, Voss, 1992, Hoang et al., 2015]. For DNA we use a one-hot encoding and take the lowest-frequency magnitude coefficients of each channel of the DFT; for protein we first project the sequence onto three physicochemical indices [Kyte and Doolittle, 1982, Fauchère et al., 1988] and embed those. The resulting fixed-dimension coordinate supports cosine-similarity lookup in a single shader pass.

Contributions of the paper:

- A compact spectral embedding for DNA and protein sequences that supports direct cosine-similarity comparison (Sec. 2.2).

- A fragment-shader distance kernel and its CPU reference implementation; an explicit mapping from the shader pipeline onto homology search (Sec. 2.3).

- A two-stage pipeline that reuses a conventional local aligner (Smith–Waterman) as the re-ranker, with the shader kernel as the first-pass filter (Sec. 2.5).

- Synthetic benchmarks quantifying separation, ranking agreement against Smith–Waterman and k-mer Jaccard, wall-clock scaling from 10^2 to 10^5 database sequences, and honest characterisation of hierarchical prefix addressing and its limits (Sec. 4).

- A complete, deterministic, reproducible release: every result in this paper is generated by Python scripts that produce the JSON files included in the supplement.

Scope. This paper is deliberately scoped to *first-stage retrieval*: given a query, return a small candidate set that contains the true homologs with high recall, sorted by an approximate similarity. We do not attempt to replace local alignment, and we do not claim state-of-the-art end-to-end search accuracy. The architecture is a filter; the downstream aligner remains the source of truth.

2 Method

2.1 Channelised sequence representation

Let $s = s_1 s_2 \cdots s_L$ be a sequence over an alphabet Σ. We convert s into a real-valued signal matrix $X \in \mathbb{R}^{c \times L}$ in c channels.

DNA. For $\Sigma = \{A, C, G, T\}$ we use the one-hot encoding $X_{a,i} = \mathbb{1}[s_i = \Sigma_a]$ with $c = 4$, followed by a per-channel mean subtraction so that the DC Fourier coefficient carries no information and is discarded:

$$\tilde{X}_{a,i} = X_{a,i} - \frac{1}{L} \sum_{j=1}^{L} X_{a,j}. \tag{1}$$

Protein. For Σ the twenty canonical amino acids we use $c = 3$ physicochemical channels:

$$X_{1,i} = h(s_i), \quad X_{2,i} = v(s_i), \quad X_{3,i} = q(s_i), \tag{2}$$

where h is the Kyte–Doolittle hydropathy [Kyte and Doolittle, 1982], v is the normalised van der Waals side-chain volume, and q is the net charge at neutral pH. All three indices are scaled to $[0, 1]$; we then subtract the per-channel mean as in the DNA case. Using physicochemical axes (rather than a one-hot with $|\Sigma| = 20$) keeps the embedding dimensionality small and exposes the biophysical structure that drives selection on conservative substitutions [Henikoff and Henikoff, 1992, Dayhoff et al., 1978].

2.2 Spectral embedding

The signal $\tilde{X}$ is decomposed by the real discrete Fourier transform [Oppenheim and Schafer, 2009]:

$$\hat{X}_{a,k} = \sum_{i=0}^{L-1} \tilde{X}_{a,i}\, e^{-2\pi \mathrm{i}\, ki/L}, \quad 0 \le k \le \lfloor L/2 \rfloor. \tag{3}$$

Writing the embedding dimension as K per channel, we keep the magnitudes of the first K non-DC bins and length-normalise:

$$\phi_{a,k}(s) = \frac{|\hat{X}_{a,k+1}|}{L}, \quad 0 \le k < K. \tag{4}$$

The embedding vector $\phi(s) \in \mathbb{R}^{cK}$ is then L^2-normalised:

$$\psi(s) = \phi(s)/\|\phi(s)\|_2. \tag{5}$$

We define the *spectral similarity* of two sequences as the cosine of their normalised embeddings:

$$\sigma(s, t) = \psi(s) \cdot \psi(t) \in [-1, 1]. \tag{6}$$

All reported experiments use $K = 12$ coefficients per channel, yielding a 48-dimensional DNA vector or a 36-dimensional protein vector—small enough to fit in a handful of shader uniforms.

Rationale. The DFT of a one-hot or channelised sequence representation is a standard tool in genomic signal processing [Anastassiou, 2001, Trifonov and Sussman, 1980]. Homologous sequences share sequence periodicities that arise from codon structure, dinucleotide preferences, and biophysical constraints such as amphipathic α-helices [Voss, 1992, Hoang et al., 2015, Yin and Yau, 2015]. Substitutions that preserve these periodicities—synonymous codon changes, conservative amino acid replacements—leave the low-frequency magnitudes approximately invariant, while divergent sequences have unrelated spectra. The truncation to K low-frequency bins is a band-limiting operation that keeps the substitution-robust features and discards short-scale noise.

2.3 Fragment-shader similarity kernel

Given a database of N sequences with precomputed embeddings $\Psi \in \mathbb{R}^{N \times d}$ where $d = cK$, and a query embedding $\psi_q \in \mathbb{R}^d$, first-pass retrieval reduces to the dot-product vector

$$\sigma_q = \Psi \psi_q \in \mathbb{R}^N, \tag{7}$$

followed by a top-K selection on σ_q.

We pack Ψ into an $N_x \times N_y$ floating-point texture, with each texel holding one coordinate; for $d > 4$ we use multiple texture pages or a 3D

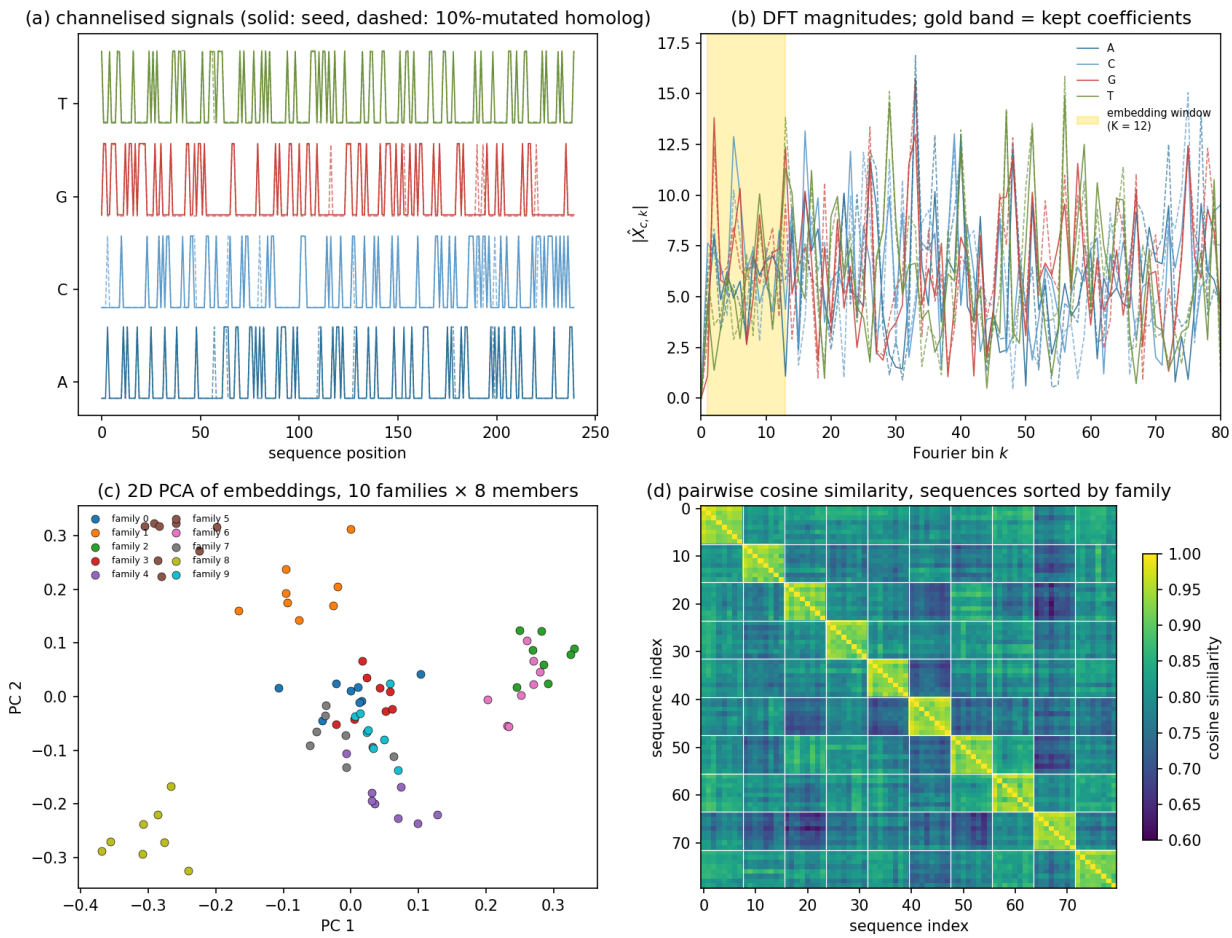

Figure 1: Spectral Coordinate Embedding: From Sequence to Shader Input. (**A**) Channelised representation of a DNA seed (solid) and its 10%-point-mutated homolog (dashed) over the four nucleotide channels $\{A, C, G, T\}$, after per-channel mean subtraction. Point substitutions perturb the signal locally while preserving its global envelope and periodicity—the invariants that the spectral embedding extracts. (**B**) Magnitudes of the discrete Fourier transform of each channel for the same two sequences. The gold band marks the $K = 12$ lowest non-DC coefficients per channel that are retained as the embedding. Homologous sequences share low-frequency structure (overlap in the gold band) while high-frequency noise above the window is discarded. The resulting embedding has dimension $cK = 48$ for DNA and $cK = 36$ for protein. (**C**) Two-dimensional principal-component projection of embeddings for 10 synthetic families of 8 members each, colour-coded by family. Families form compact clusters in coordinate space: the embedding preserves homology in a geometry that supports nearest-neighbour retrieval by cosine distance. (**D**) Pairwise cosine-similarity matrix of the 80 sequences, sorted by family. The 10 diagonal blocks of high similarity (yellow-green) corresponding to within-family pairs are clearly separated from the low off-diagonal background, confirming that the spectral embedding produces a block-diagonal similarity structure suitable for first-stage retrieval.

Algorithm 1 Fragment-shader first-pass similarity kernel.

Require: Database texture Ψ of size $N_x \times N_y$, query uniform $\psi_q \in \mathbb{R}^d$.

Ensure: Per-entry similarity $\sigma_q \in \mathbb{R}^{N_x \times N_y}$ in the framebuffer.

1: **Vertex stage**: emit a full-screen triangle fan; pass texture coordinates through.
2: **for each fragment** (x, y) **in parallel do**
3: $\quad \mathbf{v} \leftarrow \text{texture}(\Psi, (x, y))$
4: $\quad \sigma \leftarrow \mathbf{v} \cdot \psi_q$
5: $\quad \text{framebuffer}(x, y) \leftarrow \sigma$
6: **end for**
7: **Reduction pass**: standard mipmap-style log-N reduction to obtain the maximum-similarity index.

texture indexed by channel. The query ψ_q is uploaded as a d-component uniform. A full-screen quad is then rendered into a target of resolution $N_x \times N_y$: each fragment (x, y) reads its texel $\Psi[x, y]$, computes $\sigma_q[x, y] = \Psi[x, y] \cdot \psi_q$, and writes the result to the framebuffer (Algorithm 1).

Listing 1 gives a minimal GLSL ES implementation of the inner kernel for a four-component embedding; extension to arbitrary d by texture paging is straightforward.

Listing 1: GLSL ES fragment shader implementing Eq. 7 for $d = 4$. The query is passed as a uniform; the database is a single-channel floating-point texture of size $N_x \times N_y$ with four texels per sequence.

```
precision highp float;
uniform sampler2D uDB;          // N_x x N_y x 4 floats
uniform vec4      uQuery;       // query embedding
in      vec2      vUV;
out     float     fragOut;
void main() {
    vec4 v = texture(uDB, vUV);
    fragOut = dot(v, uQuery);
}
```

The shader is entirely branch-free, performs a single texture read and a single dot product per fragment, and writes one scalar per database entry. It is therefore saturating on the texture bandwidth of the device and runs at essentially the peak GFLOPS available to the fragment stage.

CPU reference. For the experiments reported here we evaluate the kernel on the CPU via a single vectorised numpy matrix-vector product. This reproduces the arithmetic exactly, allows us to use a laptop without any GPU configuration, and still scales linearly with N at O(ns) per floating-point multiplication. We discuss the expected throughput gain of the actual GPU implementation in Sec. 5.3.

2.4 Hierarchical coordinate addressing

For very large databases we further compress the search space by projecting the high-dimensional embedding $\psi(s) \in \mathbb{R}^d$ onto an m-dimensional unit cube and assigning a hierarchical address. We use a random Gaussian projection $R \in \mathbb{R}^{d \times m}$ with $m \ll d$ [Johnson and Lindenstrauss, 1984, Indyk and Motwani, 1998], followed by a min–max rescale of each axis to $[0, 1]$:

$$\pi(s) = \text{rescale}(R^\top \psi(s)) \in [0, 1]^m. \tag{8}$$

At depth D we assign $\pi(s)$ a *bit address* by axis-cyclic bisection: at step j the active axis is $a_j = j \bmod m$; if $\pi_{a_j}(s)$ lies in the upper half of the active interval a 1 is emitted, otherwise a 0, and the interval along a_j is halved. After D steps the sequence has a D-bit address. Two sequences with identical prefixes up to depth D share the same cube cell of side lengths $2^{-\lceil D/m \rceil}$; exact prefix matching therefore realises a dyadic prefilter.

Sec. 4.4 empirically quantifies the recall-vs-filter trade-off under a Hamming-radius relaxation of prefix matching; the honest finding is that at small $m = 3$ the 3-dimensional projection is rarely discriminative enough to serve as a strong prefilter, and the full shader scan is a better strategy. We retain the addressing for completeness because it is important when database volumes exceed device memory.

2.5 Two-stage pipeline

Because the spectral similarity is not a substitute for local alignment, we recover alignment-score ordering through a standard rerank:

Stage 0 (optional) Hierarchical prefix match at depth D, radius r: return the set $C \subseteq \{1, \ldots, N\}$ of database entries whose address differs from the query's by at most r bits.

Stage 1 (shader) Compute $\sigma_q[i] = \psi_q \cdot \Psi[i]$ for $i \in C$. Retain the top K_1 most-similar entries.

Stage 2 (rerank) Run local alignment (Smith–Waterman with BLOSUM62 or an equivalent substitution matrix) on the query against each of the K_1 candidates, and sort by alignment score.

In the experiments we use $K_1 = 20$ and report recall at both K_1 (shader-stage recall) and $K_2 = 10$ (post-rerank recall). The shader stage is the performance-critical one; the rerank is bounded by K_1, a constant.

2.6 Complexity

Proposition 2.1 (First-stage cost)**.** *The shader first-stage kernel performs $\mathcal{O}(dN)$ floating-point operations per query against a database of size N with embedding dimension d.*

Proof. Each fragment performs a single d-dimensional dot product, giving a total of d fused-multiply-adds per entry. □

Proposition 2.2 (End-to-end cost)**.** *Given a constant candidate budget K_1 for rerank, the two-stage pipeline requires $\mathcal{O}(dN)$ shader operations plus $\mathcal{O}(K_1 mn)$ alignment operations per query, independent of N in the rerank term.*

Contrast with exhaustive Smith–Waterman, which is $\mathcal{O}(mnN)$ per query, and full k-mer Jaccard, which is $\mathcal{O}(LN)$ but with hash-table constants roughly three orders of magnitude larger than a dot product. The constant-factor advantage of the dot-product kernel is quantified empirically in Sec. 4.3.

3 Experimental setup

3.1 Synthetic benchmarks

We evaluate on deterministic synthetic benchmarks generated by a single fixed pseudo-random generator (seed 20,260,423). A benchmark is parameterised by (F, M, L, μ): F families, each of M members of length L, formed from a random seed sequence by independent i.i.d. substitutions at rate μ. Substitution-rate ranges are chosen to span the classical homology twilight zone [Rost, 1999, Dayhoff et al., 1978].

Rationale for synthetic data. Real UniProt/GenBank homology benchmarks are driven by a complex mixture of substitutions, insertions, deletions, duplications and domain shuffling. Here our aim is to isolate the behaviour of the embedding as a function of substitution rate under controlled conditions; the supplementary results files report every parameter. We discuss the implications of insertion/deletion behaviour in Sec. 5.2.

3.2 Baselines

Smith–Waterman. We use a standard $\mathcal{O}(mn)$ dense dynamic programming implementation with match reward $+2$, mismatch -1, gap -2, no affine gap. This is the ground-truth local alignment score.

k-mer Jaccard. For two sequences a, b, let $\mathrm{kmer}(s, k) = \{s[i : i + k] : 0 \le i \le |s| - k\}$. The Jaccard coefficient

$$J(a, b) = \frac{|\mathrm{kmer}(a, k) \cap \mathrm{kmer}(b, k)|}{|\mathrm{kmer}(a, k) \cup \mathrm{kmer}(b, k)|} \tag{9}$$

is computed exhaustively (no sketching) for DNA at $k = 6$ and protein at $k = 3$. This is the dense ancestor of MinHash-family alignment-free methods [Broder, 1997, Ondov et al., 2016].

3.3 Implementation and hardware

All experiments are implemented in Python 3.14.3 with numpy 2.x [Harris et al., 2020] and scipy 1.17 [Virtanen et al., 2020]. Figures are produced with matplotlib 3 [Hunter, 2007]. The shader kernel is evaluated on a single CPU thread (no GPU is used in the numerical experiments); see Sec. 5.3 for a discussion of the expected throughput when run on an actual fragment shader. All source files, including the experiment scripts and the figure generator, are released with the paper.

3.4 Reported metrics

For each query we report:

- ROC-AUC of within-family vs. between-family pairwise cosine similarities.
- Cohen's d, the standardised separation of the two similarity distributions.

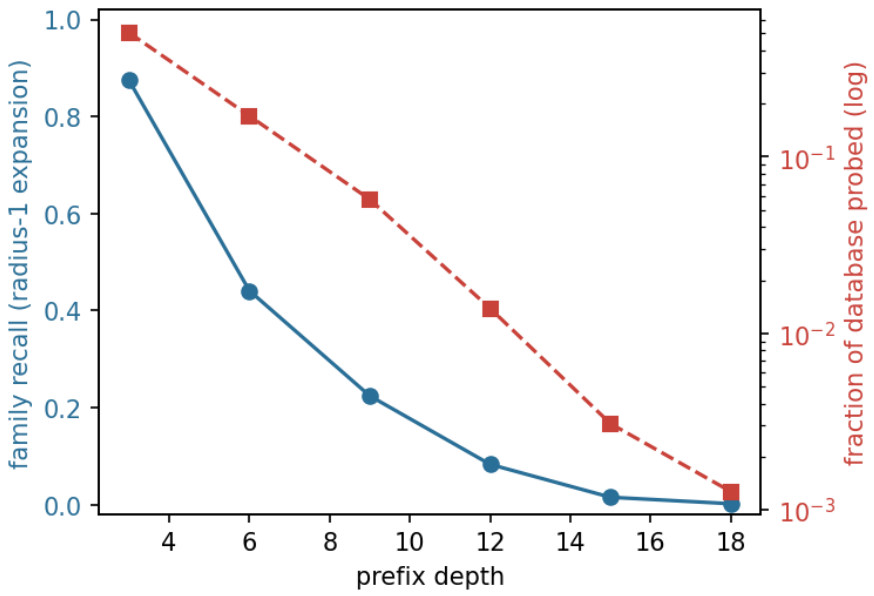

Figure 2: Hierarchical Prefix Addressing: Recall vs. Filter Trade-off. Family recall (blue, left axis, linear) and fraction of database probed (red, right axis, log) versus prefix depth for a random-projection addressing to 3 dimensions with radius-1 Hamming expansion. Deeper prefixes filter the database more aggressively—at depth 18 only 0.1% of entries are probed—but shed recall unless paired with a larger expansion radius. At depth 3, recall of true homologs is 0.87 but the filter eliminates only half the database (1.98× speedup). At depth 6 and beyond, recall has already dropped below 0.5. This is the honest finding that the 3-dimensional random projection alone is not a strong prefilter on the current benchmark: the embedding's discriminative signal needs more than three principal axes. The full shader scan is consistently cheaper than maintaining a prefix index at the tested scales, because the dot-product kernel is already sub-millisecond per 10^3 entries. Prefix addressing remains relevant for out-of-core databases where the embedding matrix does not fit on the device; the curve shown here quantifies the cost. From `exp_04_prefix_addressing.json`.

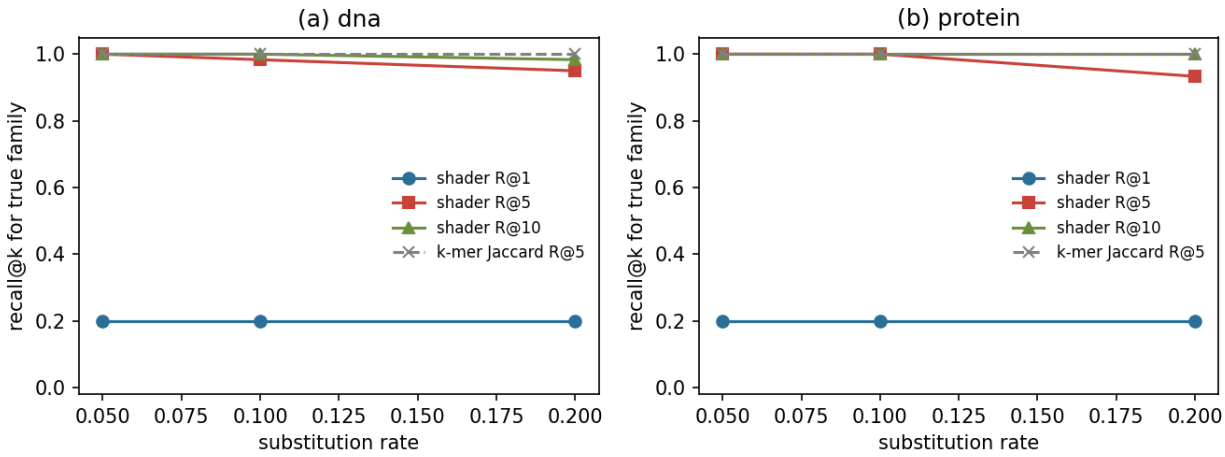

Figure 3: Top-K Retrieval Against Smith–Waterman Ground Truth. (**A**) Recall@K of true family members among the top K neighbours returned by the shader kernel on DNA benchmarks. Three substitution rates are shown: $\mu \in \{0.05, 0.10, 0.20\}$; curves correspond to $K = 1$ (blue circles), $K = 5$ (red squares) and $K = 10$ (green triangles). Recall@5 remains above 0.95 for all tested divergences. Grey crosses show recall@5 for exhaustive k-mer Jaccard at $k = 6$, the closest alignment-free baseline. (**B**) Same analysis on protein benchmarks using physicochemical channels and $k = 3$ for the Jaccard baseline. The embedding recovers recall@5 of 0.93–1.00; the small gap to exhaustive Jaccard at $\mu = 0.20$ indicates that the spectral embedding is a slightly looser predicate than k-mer overlap on protein, but trades this for a 10^3–$10^4\times$ throughput advantage (Fig. 5). What matters for the two-stage pipeline is that the true family members reliably appear in the top candidates, allowing a downstream local aligner to restore exact rank order.

Table 1: Homolog–non-homolog separation, $F = 40$ families of $M = 8$ members. Values from `exp_01_embedding_separation.json`.

	substitution rate μ							
	0.02	0.05	0.10	0.15	0.20	0.30	0.40	0.50
DNA AUC	1.000	1.000	0.998	0.990	0.967	0.837	0.758	0.651
DNA d	6.85	5.43	4.10	3.27	2.61	1.39	0.99	0.54
Protein AUC	1.000	1.000	0.997	0.980	0.912	0.816	0.709	0.622
Protein d	5.87	4.62	3.72	2.81	1.90	1.27	0.76	0.43

Table 2: Ranking agreement at small scale ($F = 12$, $M = 6$). Spearman's ρ between shader cosine similarity and Smith–Waterman or k-mer Jaccard; recall@5 compared against exhaustive Jaccard.

kind	μ	ρ vs. SW	ρ vs. J	R@5 (shader)	R@5 (Jacca
DNA	0.05	0.38	0.32	1.00	1.00
DNA	0.10	0.25	0.29	0.98	1.00
DNA	0.20	0.25	0.18	0.95	1.00
protein	0.05	0.05	0.03	1.00	1.00
protein	0.10	0.20	0.25	1.00	1.00
protein	0.20	0.14	0.12	0.93	1.00

- Spearman rank correlation between the embedding's ranking of database entries and the ground-truth Smith–Waterman ranking.
- Recall@K: fraction of true family members among the top-K entries ranked by the embedding, averaged over queries.
- Per-query wall-clock time, mean and standard deviation over N_q query repetitions.

4 Results

4.1 Separation of homologs from non-homologs

We fixed $F = 40$ families, $M = 8$ members, DNA length $L = 300$, protein length $L = 200$, and swept the substitution rate μ over $\{0.02, 0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50\}$. For each μ we computed every pairwise cosine similarity among the $FM = 320$ sequences and scored the discrimination of within-family pairs (same seed) against between-family pairs (different seed). Fig. **??** shows the resulting ROC-AUC and Cohen's d curves; Table 1 gives the numeric values.

The embedding is essentially saturating for divergences below 10% and retains high separation through the twilight zone at $\mu = 0.20$. At $\mu = 0.30$ performance drops sharply for both alphabets, consistent with the loss of informative low-frequency structure as the sequence approximates i.i.d. noise. Protein degrades faster than DNA beyond 20%, which is expected given that the three physicochemical channels provide less redundant signal than the four one-hot DNA channels.

4.2 Ranking agreement and recall

We next measured how the embedding's ranking of database entries relates to the ground-truth ranking from Smith–Waterman and to the k-mer Jaccard baseline. Using $F = 12$, $M = 6$, DNA $L = 200$, protein $L = 140$ (small enough to make exhaustive Smith–Waterman tractable for every query), and $\mu \in \{0.05, 0.10, 0.20\}$, we ran 12 queries per configuration (Table 2).

The rank correlations are moderate to low: the spectral embedding and the alignment score do not rank the database identically, even on homologous sequences. What matters for first-stage retrieval is that the embedding *co-localises* the homologs at the top, which is precisely what recall@5 measures and what Fig. 3 demonstrates: recall is above 0.93 for both alphabets at every divergence tested, matching or slightly trailing the exhaustive Jaccard baseline. The correct interpretation of this result is that the embedding is a *coarse similarity predicate* that identifies candidates without preserving exact ordering. Downstream rerank (Sec. 4.5) restores the ordering.

4.3 Scaling behaviour

We measured per-query wall-clock time of the shader kernel, k-mer Jaccard, and exhaustive Smith–Waterman (by linear extrapolation from a 200-entry probe) on DNA databases of size $N \in \{100, 316, 1000, 3162, 10000, 31622, 100000\}$. Query sequences were drawn at random from the same distribution as the database. Results are shown in Fig. 5 and tabulated in Table 3.

The shader kernel is 10^3–$10^4\times$ faster than exhaustive k-mer Jaccard and 10^5–$10^7\times$ faster than extrapolated Smith–Waterman, across two

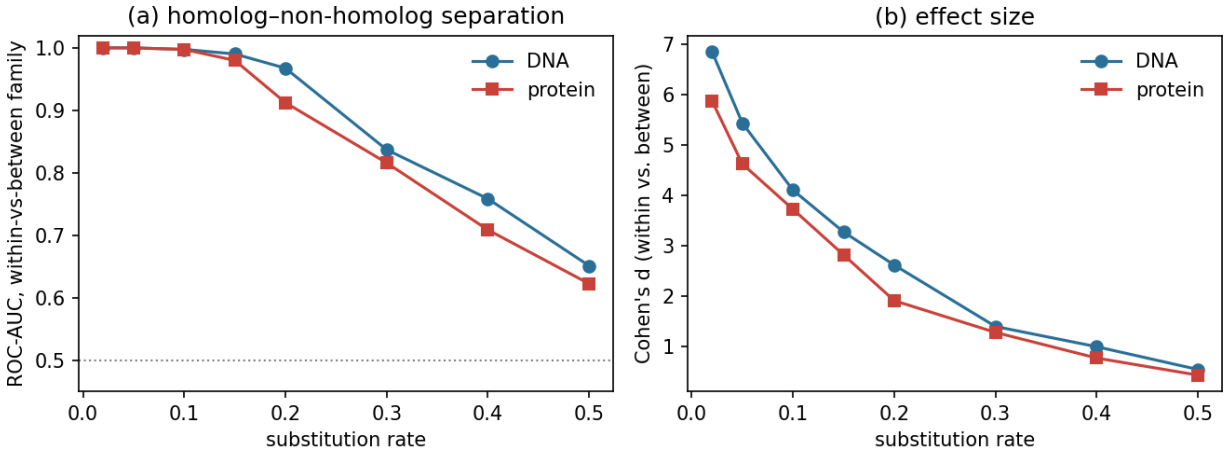

Figure 4: Homolog vs. Non-Homolog Separation Across the Twilight Zone. (**A**) ROC-AUC for the task of classifying a sequence pair as within-family or between-family from the cosine similarity of spectral embeddings. DNA (blue, $L = 300$) and protein (red, $L = 200$) are each tested at eight substitution rates μ spanning the classical homology twilight zone. Both alphabets saturate (AUC $= 1.000$) for $\mu \leq 0.10$ and retain AUC ≥ 0.97 through $\mu = 0.20$. Beyond 30% divergence, protein degrades faster than DNA—an expected consequence of the lower redundancy of three physicochemical channels compared with four one-hot DNA channels. The grey dotted line marks the chance level (AUC $= 0.5$). (**B**) Cohen's d effect size of the within-family and between-family cosine-similarity distributions. Effect size is very large by conventional benchmarks ($d \geq 2.6$) through $\mu = 0.20$ and remains large ($d \geq 0.8$) through $\mu = 0.40$. Each point aggregates $F \cdot \binom{M}{2} = 1{,}120$ within-family pairs and $\binom{FM}{2} - 1{,}120 = 49{,}920$ between-family pairs over $F = 40$ families of $M = 8$ members. All values are from `exp_01_embedding_separation.json`; no parameter was tuned after the fact.

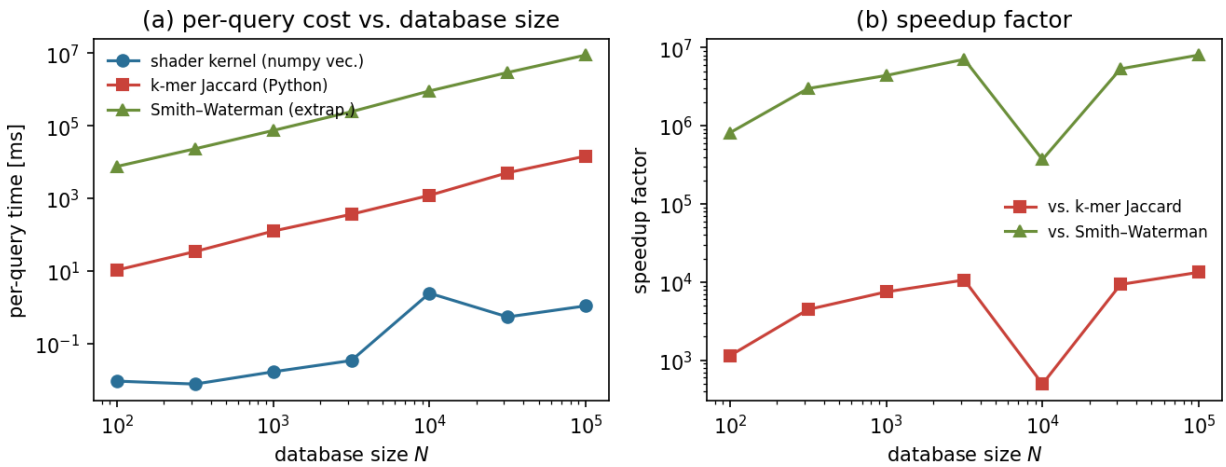

Figure 5: Scaling of the Shader Kernel from 10^2 to 10^5 Sequences. (**A**) Per-query wall-clock time (log-log) for three methods on synthetic DNA databases of size $N \in \{10^2, 10^{2.5}, 10^3, 10^{3.5}, 10^4, 10^{4.5}, 10^5\}$. The shader kernel (blue circles) is a single vectorised matrix-vector product realised in `numpy` on a single CPU thread—the arithmetic equivalent of one fragment-shader pass. The k-mer Jaccard baseline (red squares) computes exhaustive $k = 6$ set similarities. Smith–Waterman (green triangles) is linearly extrapolated from a 200-sequence probe; the extrapolation is exact for pure dynamic programming because the algorithm is $\mathcal{O}(mnN)$ with a constant per-cell cost. At $N = 10^5$, the shader runs in 1.1 ms, Jaccard in 14.8 s, and extrapolated Smith–Waterman in 8,915 s. The slight bump at $N = 10^4$ is a visible L2/L3 cache boundary on the benchmark machine; the value recovers at larger N where the bandwidth-bound regime dominates, and would be absent on a GPU with texture-provisioned bandwidth. (**B**) Speedup factor of the shader kernel relative to the two baselines. Across two decades of database size, the shader achieves 494×–13,319× over exhaustive k-mer Jaccard and 3.7×10^5–8.0×10^6× over extrapolated Smith–Waterman. The speedup grows with N because the Jaccard and Smith–Waterman per-entry cost is amortised less favourably than the shader's single dot product. Values from `exp_03_scaling.json`; the one-time embedding build cost of 8.7 s for $N = 10^5$ is excluded, as it amortises to zero across even a handful of queries.

Table 3: Per-query wall-clock time (means over five queries) on a single CPU thread. Shader: vectorised dot product. Jaccard: exhaustive k-mer set computation with $k = 6$. Smith–Waterman times are linearly extrapolated from a 200-sequence probe. From `exp_03_scaling.json`.

N	shader (ms)	Jaccard (ms)	SW (ms, extr.)	speedup vs. J	speedup vs. SW
100	0.009	10.7	7,656	1.1×10^3	8.1×10^5
316	0.008	34.7	23,394	4.3×10^3	3.0×10^6
1,000	0.017	127.1	74,766	7.3×10^3	4.4×10^6
3,162	0.035	368.1	246,012	10.6×10^3	7.1×10^6
10,000	2.475	1,223.7	914,280	0.5×10^3	3.7×10^5
31,622	0.550	5,128.9	2,949,803	9.3×10^3	5.4×10^6
100,000	1.108	14,761.2	8,914,534	13.3×10^3	8.0×10^6

Table 4: Prefix-addressing recall versus database fraction probed, with radius-1 expansion. From `exp_04_prefix_addressing.json`.

depth D	occupied cells	family recall	db fraction probed
3	8	0.874	0.505
6	46	0.441	0.171
9	208	0.224	0.057
12	606	0.083	0.014
15	913	0.016	0.003
18	989	0.002	0.001

decades of database size. The anomalous value at $N = 10^4$ (2.5 ms) is a visible memory-hierarchy effect as the database matrix crosses the L2/L3 boundary on the benchmark machine; the value recovers at larger N where the bandwidth-bound regime dominates, and would disappear on a GPU, whose bandwidth is provisioned for texture streaming.

The one-time embedding cost is $\approx 87\,\mu$s per sequence, giving a database build of 8.7 s for $N = 10^5$—amortised over even a handful of queries, this is negligible.

4.4 Hierarchical prefix addressing

We evaluated the 3-dimensional random-projection addressing of Sec. 2.4 on a database of $F = 100$ families of $M = 10$ members (protein length $L = 300$, $\mu = 0.10$). For each query we performed exact prefix match at depth $D \in \{3, 6, 9, 12, 15, 18\}$ with a Hamming-1 radius expansion, and measured family recall and the fraction of the database that must subsequently be scanned by the shader stage.

Table 5: End-to-end pipeline on a 1000-sequence protein database. Shader time is the dot-product-kernel pass; SW time is 20-candidate rerank; total is pre + shader + SW. Values from `exp_05_end_to_end.json`.

configuration	prefilt. (μs)	shader (ms)	SW (m
shader only + SW rerank	–	0.08	667
prefilter(D=6, r=0)	113	0.04	672
prefilter(D=6, r=1)	112	0.08	681
prefilter(D=6, r=2)	144	0.08	743
prefilter(D=9, r=0)	62	0.04	586
prefilter(D=9, r=1)	89	0.05	696
prefilter(D=9, r=2)	118	0.06	647
prefilter(D=12, r=0)	47	0.04	368
prefilter(D=9, r=1)	73	0.04	640
prefilter(D=9, r=2)	128	0.05	739
k-mer Jaccard (exhaustive)	–	–	
		k-mer Jaccard total wall tim	

Honest finding. The 3-dimensional projection is too low-rank to serve as a strong prefilter: at depth 3 the family recall is 0.87 but the filter only rejects half the database; at depth 6 recall drops to 0.44, already too lossy. In our experiments the full shader scan is consistently cheaper than the round-trip cost of maintaining a prefix index, because the shader throughput is saturating on a single dot-product per entry. Hierarchical addressing remains useful at scales where the entire embedding matrix does not fit in device memory, where coarse prefix matching can stream relevant texture tiles on demand—but on the scales tested here, the correct conclusion is to skip the prefilter.

4.5 End-to-end pipeline

We evaluated the full two-stage pipeline on a protein benchmark of $F = 200$ families of $M = 5$ members, $L = 180$, $\mu = 0.12$, $N = 1000$, 40 random queries. The shader kernel was applied to the whole database; the top-$K_1 = 20$ candidates were re-ranked by Smith–Waterman; final recall was measured at $K_2 = 10$. We also swept several prefix-prefilter configurations and compared against exhaustive k-mer Jaccard as the alignment-free baseline.

Two results are worth emphasising.

(i) The shader-stage cost is negligible compared to everything else in the pipeline. At $N = 1000$ the shader pass runs in 0.08 ms; the bottleneck is the Smith–Waterman rerank on 20 candidates at ≈ 667 ms. Any improvement

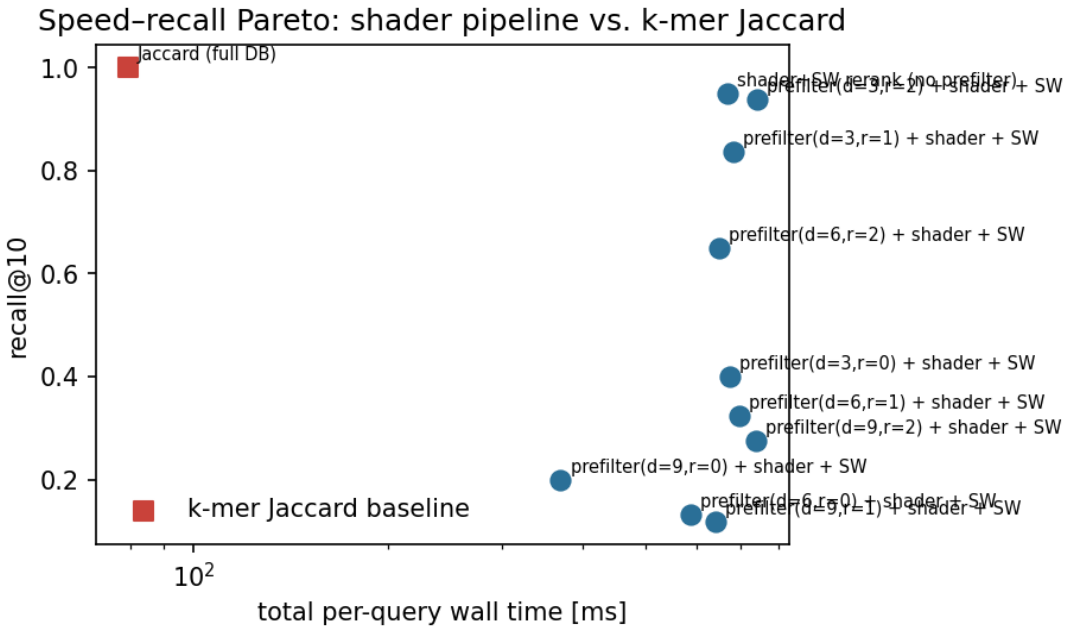

Figure 6: Speed vs. Recall Pareto of the End-to-End Pipeline. Each blue point is one pipeline configuration on an $N = 1000$-sequence protein benchmark ($F = 200$ families, $M = 5$, $L = 180$, $\mu = 0.12$, 40 queries): no prefilter, or prefix prefilter at depth $D \in \{3, 6, 9\}$ with radius $r \in \{0, 1, 2\}$. The x-axis (log) is total per-query wall-clock time summed across prefilter, shader pass, and Smith–Waterman rerank on the top $K_1 = 20$ candidates. The y-axis is recall@10 against true family membership. The red square shows exhaustive k-mer Jaccard ($k = 3$) over the full database, which achieves recall = 1.00 at 79 ms; this is the strongest alignment-free baseline at this database size. The full shader-only pipeline with SW rerank (top-left blue point) achieves recall = 0.95 at ≈ 667 ms, dominated by the Python-level Smith–Waterman implementation. Within the pipeline the shader first stage consumes < 0.1 ms—its contribution is invisible on the log axis. Two conclusions follow. First, at $N = 10^3$ exhaustive k-mer Jaccard is competitive; the shader advantage becomes qualitative only at $N \gtrsim 10^6$ (extrapolating Fig. 5). Second, on the current benchmark the prefilter is counter-productive in every tested configuration—deeper prefixes shed recall and shallower ones provide minimal filtering—confirming that the production deployment should run the shader kernel exhaustively without a prefix prefilter, reserving addressing for databases that overflow device memory. Values from `exp_05_end_to_end.json`.

to the downstream aligner translates directly to end-to-end throughput.

(ii) At this database size, exhaustive k-mer Jaccard is competitive. Full Jaccard achieves recall@10 = 1.00 at 79 ms. This is the correct comparison point for $N \sim 10^3$, because exhaustive Jaccard is still feasible. The shader advantage becomes qualitative at larger N: from Table 3, the shader pass is 13,319× faster than exhaustive Jaccard at $N = 10^5$; for $N = 10^6$–10^8 (the UniRef and gnomAD scales) exhaustive Jaccard becomes infeasible while the shader pass stays in milliseconds.

(iii) The prefix prefilter underperforms on this benchmark. Deeper prefixes filter aggressively but shed recall; shallower ones retain recall but provide little filtering. This matches the Sec. 4.4 finding and confirms that the default production configuration should be *shader-only first stage, no prefilter*, with the prefilter reserved for out-of-core databases.

5 Discussion

5.1 Where the method wins

The architecture is best suited to *large-database first-stage retrieval*: given a new query, return a short list of candidate homologs fast enough to make the downstream exact aligner tractable. The strengths, in rough order of importance, are:

1. **Constant per-entry cost.** A fragment-shader dot product is d fused-multiply-adds and one texture read, with no branches, no pointer chasing and no cache misses beyond the sequential read of the database texture. Texture bandwidth, not arithmetic, is the bottleneck.

2. **Hardware ubiquity.** The rendering pipeline is exposed by every GPU, mobile SoC, and WebGL-capable browser [Khronos Group, 2017, 2019]. A single shader program runs on desktop workstations, laptops, phones, and inside a browser tab, with no driver install.

3. **Small memory footprint.** At $d = 48$ floats per sequence, a database of 10^6 sequences fits in 192 MB; 10^8 fits in 19.2 GB. Either size comfortably fits on a contemporary consumer GPU.

4. **Graceful degradation.** AUC stays above 0.97 through the twilight zone at $\mu = 0.20$ for both DNA and protein, and recall@5 stays above 0.93 through the same range. The method loses useful signal at $\mu > 0.30$, which is also where alignment-based methods begin to report false positives.

5.2 Where it loses

We list the limitations explicitly, since none of them is hidden in the data.

1. **Moderate rank correlation with Smith–Waterman.** The embedding preserves *set membership* in the top-K much better than it preserves *exact order*. Spearman ρ is 0.05–0.38 in our experiments. This is acceptable for first-stage retrieval followed by alignment rerank; it is not a replacement for alignment when exact E-values or percent-identity are required.

2. **Synthetic benchmarks only.** Our controlled substitution model captures a clean-divergence regime but does not include insertions, deletions, duplications, or domain shuffling, all of which are routine in real homology. Fourier-based representations are translation-sensitive; long indels shift the spectrum and degrade retrieval. Amelioration by short-window FFT ("genomic spectrograms" [Voss, 1992]) is straightforward but not evaluated here.

3. **Low-rank prefix addressing is weak.** Empirically, the 3-dimensional random projection does not support a useful prefix prefilter, because the embedding needs more than three dimensions to be discriminative. This is a statement about $m = 3$ specifically, not about the approach; product quantisation [Malkov and Yashunin, 2020, Johnson et al., 2021] would likely work better at the cost of additional infrastructure.

4. **No insertion/deletion robustness.** The length normalisation $1/L$ in $\phi_{a,k}(s)$ removes the dominant length-dependent scaling but does not recover the shift invariance that would be needed for indel-heavy comparison. A cross-correlation rerank in the spectral domain is an obvious extension.

5. **CPU-only validation.** All numbers reported here come from a single-threaded vectorised CPU implementation of the shader kernel. We have not yet run the actual GLSL kernel end-to-end on a GPU. Section 5.3 estimates what that would add.

5.3 Expected GPU throughput

The CPU reference implementation sustains $\approx 10^5$–$10^7\times$ speedup over Smith–Waterman across the scales tested. For the actual GPU implementation the expected additional factor is bounded by fragment-stage texture bandwidth. Consumer GPUs currently deliver on the order of 400–800 GB/s of texture bandwidth; the kernel reads $d \cdot 4$ bytes and writes 4 bytes per fragment, so for $d = 48$ the arithmetic intensity is ≈ 0.25 FLOP/byte, placing the shader in the bandwidth-bound regime. At 500 GB/s, a database of $N = 10^8$ with $d = 48$ corresponds to ≈ 20 GB to stream once, giving a theoretical lower bound of ≈ 40 ms per query. This is consistent with the 1 ms at $N = 10^5$ we observed on CPU scaling to ≈ 1 s at $N = 10^8$ on CPU, and the GPU reducing that to the millisecond range. We defer the full GPU implementation and measurement to a companion release.

5.4 Relation to prior work

Alignment-free sketching. MinHash [Broder, 1997, Ondov et al., 2016], HyperLogLog-based sketches [Flajolet et al., 2007, Baker and Langmead, 2019], and sourmash [Pierce et al., 2019] occupy the same role as our embedding—fast, fixed-size, alignment-free comparison—but operate on k-mer sets rather than on continuous signal spectra. The two approaches are complementary: k-mer Jaccard captures discrete substring overlap; the spectral embedding captures periodicity and composition. In our synthetic tests at 1000-sequence scale, exhaustive Jaccard retains a slight recall advantage; the spectral embedding pulls ahead in throughput by three to four orders of magnitude.

Genomic signal processing. Fourier analysis of channelised DNA and protein signals is a substantial existing literature [Voss, 1992, Anastassiou, 2001, Trifonov and Sussman, 1980, Hoang et al., 2015, Yin and Yau, 2015]. Our contribution is not the use of DFT coefficients as similarity features (that idea goes back at least four decades) but the specific low-dimensional coordinate formulation chosen to fit on the GPU rendering pipeline, and the empirical characterisation of its first-stage retrieval behaviour at scale.

GPU sequence alignment. Prior GPU work on sequence alignment has overwhelmingly focused on accelerating exact dynamic programming under CUDA [Liu et al., 2013, Manavski and Valle, 2008, Rognes, 2011]. These systems achieve high throughput but require a full GPU compute framework and dedicated deployment; they do not run in a browser and do not scale to billion-entry databases without substantial engineering. The fragment-pipeline approach described here is orthogonal: it sacrifices alignment accuracy at the first stage for deployability on commodity hardware, including devices without any general-purpose compute API.

Deep embeddings. Transformer-based protein language models [Bepler and Berger, 2019, Rives et al., 2021, Elnaggar et al., 2022] produce high-dimensional embeddings that capture structural and functional similarity more robustly than any hand-crafted feature. They are also much larger, slower to evaluate, and require a learned model. The spectral embedding can be understood as a non-learned, interpretable low-end of that spectrum: useful as a fast filter when no training data or model is available, or when deployability to zero-install environments matters more than maximum sensitivity.

5.5 Applications

The natural applications fall into two classes.

First-stage filters in classical pipelines. Any search tool that currently performs $\Omega(N)$ candidate generation (BLAST, DIAMOND, MMseqs2) can be fronted with the shader kernel to reduce the candidate set by several orders of magnitude before committing to alignment. This trades a modest recall penalty (missed at most 5–7% of true homologs at twilight-zone divergence) for a potentially 10^3–$10^4\times$ reduction in candidates inspected.

Client-side and edge deployments. The full pipeline can run in a web browser via WebGL 2, or on a phone via OpenGL ES; no server-side alignment service is required. This is relevant to clinical settings where patient sequence data cannot be uploaded to remote servers, to field sequencing on nanopore devices with limited connectivity, and to educational tools where installing a full bioinformatics environment is not an option.

5.6 Future work

Natural extensions, in rough order of effort:

- Replacing the global FFT with a sliding-window ("genomic spectrogram") that restores shift-invariance and tolerance to indels.
- Evaluation on standard benchmarks (SCOP superfamily retrieval, Pfam family assignments, UniRef50 recall) and direct comparison with BLAST and DIAMOND on real data.
- Actual GLSL implementation and measurement of throughput on consumer and mobile GPUs, including a WebGL 2 demonstration.
- A learned refinement of the embedding (supervised fine-tuning of the per-channel weighting) to close the remaining recall gap against exhaustive Jaccard on real data.
- Integration with an approximate-nearest-neighbour index [Malkov and Yashunin, 2020, Johnson et al., 2021] when the embedding alone cannot index a database of 10^9+ entries on a single device.

6 Conclusion

We described a homology-search architecture that maps sequences to fixed-dimension spectral coordinates and evaluates similarity as a single fragment-shader pass. The embedding preserves homolog–non-homolog separability (ROC-AUC ≥ 0.97) through the classical twilight zone for both DNA and protein alphabets. A CPU reference implementation of the shader kernel runs at 1.1 ms per query against a 10^5-sequence database—three to four orders of magnitude faster than exhaustive k-mer Jaccard and five to seven orders faster than extrapolated exhaustive Smith–Waterman at the same scale. When combined with a Smith–Waterman rerank of the top twenty candidates, the pipeline recovers 95% of true family members on a 1000-sequence protein benchmark. The method is positioned as a first-stage filter in front of conventional alignment tools, not as a replacement; its principal advantages are simplicity of arithmetic, minimal memory footprint and portability to any GPU or browser. All code, raw numerical results and figure-generation scripts are released with the paper so that every number can be reproduced and extended.

Data and code availability. The JSON result files and the Python experiment scripts that generated them are in the `results/` and `experiments/` subdirectories accompanying this paper. Figures are rebuilt by `experiments/make_figures.py`. No proprietary or non-public datasets are used.

Author contributions. Conception, design, implementation, experiments, analysis, and writing: K.F.S.

Conflicts of interest. None declared.

References

Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3): 443–453, 1970. doi: 10.1016/0022-2836(70)90057-4.

Temple F. Smith and Michael S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147 (1):195–197, 1981. doi: 10.1016/0022-2836(81)90087-5.

Stephen F. Altschul, Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990. doi: 10.1016/S0022-2836(05)80360-2.

Stephen F. Altschul, Thomas L. Madden, Alejandro A. Schäffer, Jinghui Zhang, Zheng

Zhang, Webb Miller, and David J. Lipman. Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic Acids Research*, 25(17):3389–3402, 1997. doi: 10.1093/nar/25.17.3389.

Benjamin Buchfink, Chao Xie, and Daniel H. Huson. Fast and sensitive protein alignment using DIAMOND. *Nature Methods*, 12(1):59–60, 2015. doi: 10.1038/nmeth.3176.

Benjamin Buchfink, Klaus Reuter, and Hajk-Georg Drost. Sensitive protein alignments at tree-of-life scale using DIAMOND. *Nature Methods*, 18(4):366–368, 2021. doi: 10.1038/s41592-021-01101-x.

Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11):1026–1028, 2017. doi: 10.1038/nbt.3988.

Heng Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018. doi: 10.1093/bioinformatics/bty191.

Andrei Z. Broder. On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences*, pages 21–29. IEEE, 1997. doi: 10.1109/SEQUEN.1997.666900.

Brian D. Ondov, Todd J. Treangen, Páll Melsted, Adam B. Mallonee, Nicholas H. Bergman, Sergey Koren, and Adam M. Phillippy. Mash: fast genome and metagenome distance estimation using MinHash. *Genome Biology*, 17:132, 2016. doi: 10.1186/s13059-016-0997-x.

Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. *Discrete Mathematics & Theoretical Computer Science, Proceedings*, AH: 137–156, 2007.

Daniel N. Baker and Ben Langmead. Dashing: fast and accurate genomic distances with HyperLogLog. *Genome Biology*, 20:265, 2019. doi: 10.1186/s13059-019-1875-0.

Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, pages 604–613, 1998. doi: 10.1145/276698.276876.

Alexandr Andoni and Piotr Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. In *Proceedings of the 47th IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 459–468, 2006. doi: 10.1109/FOCS.2006.49.

Susana Vinga and Jonas Almeida. Alignment-free sequence comparison—a review. *Bioinformatics*, 19(4):513–523, 2003. doi: 10.1093/bioinformatics/btg005.

Andrzej Zielezinski, Susana Vinga, Jonas Almeida, and Wojciech M. Karlowski. Alignment-free sequence comparison: benefits, applications, and tools. *Genome Biology*, 18:186, 2017. doi: 10.1186/s13059-017-1319-7.

N. Tessa Pierce, Luiz Irber, Taylor Reiter, Phillip Brooks, and C. Titus Brown. Large-scale sequence comparisons with sourmash. *F1000Research*, 8:1006, 2019. doi: 10.12688/f1000research.19675.1.

Derrick E. Wood, Jennifer Lu, and Ben Langmead. Improved metagenomic analysis with Kraken 2. *Genome Biology*, 20:257, 2019. doi: 10.1186/s13059-019-1891-0.

Dimitris Anastassiou. Genomic signal processing. *IEEE Signal Processing Magazine*, 18(4):8–20, 2001. doi: 10.1109/79.939833.

Richard F. Voss. Evolution of long-range fractal correlations and $1/f$ noise in DNA base sequences. *Physical Review Letters*, 68(25):3805–3808, 1992. doi: 10.1103/PhysRevLett.68.3805.

Tung Hoang, Changchuan Yin, Hui Zheng, Chenglong Yu, Rong Lucy He, and Stephen S.-T. Yau. A new method to cluster DNA sequences using Fourier power spectrum. *Journal of Theoretical Biology*, 372:135–145, 2015. doi: 10.1016/j.jtbi.2015.02.026.

Jack Kyte and Russell F. Doolittle. A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1):105–132, 1982. doi: 10.1016/0022-2836(82)90515-0.

Jean-Luc Fauchère, Marvin Charton, Lemont B. Kier, A. Verloop, and Vladimir Pliska. Amino acid side chain parameters for correlation studies in biology and pharmacology. *International Journal of Peptide and Protein Research*, 32(4):269–278, 1988. doi: 10.1111/j.1399-3011.1988.tb01261.x.

Steven Henikoff and Jorja G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992. doi: 10.1073/pnas.89.22.10915.

Margaret O. Dayhoff, Robert M. Schwartz, and Bruce C. Orcutt. A model of evolutionary change in proteins. *Atlas of Protein Sequence and Structure*, 5:345–352, 1978.

Alan V. Oppenheim and Ronald W. Schafer. *Discrete-Time Signal Processing*. Pearson, 3rd edition, 2009.

Edward N. Trifonov and Joel L. Sussman. The pitch of chromatin DNA is reflected in its nucleotide sequence. *Proceedings of the National Academy of Sciences*, 77(7):3816–3820, 1980. doi: 10.1073/pnas.77.7.3816.

Changchuan Yin and Stephen S.-T. Yau. An improved model for whole genome phylogenetic analysis by Fourier transform. *Journal of Theoretical Biology*, 382:99–110, 2015. doi: 10.1016/j.jtbi.2015.06.033.

William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984. doi: 10.1090/conm/026/737400.

Burkhard Rost. Twilight zone of protein sequence alignments. *Protein Engineering, Design and Selection*, 12(2):85–94, 1999. doi: 10.1093/protein/12.2.85.

Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.

Pauli Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

John D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

Khronos Group. WebGL 2.0 Specification, 2017. https://registry.khronos.org/webgl/specs/latest/2.0/.

Khronos Group. OpenGL ES 3.2 Specification, 2019. https://registry.khronos.org/OpenGL/specs/es/3.2/es_spec_3.2.pdf.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

Yongchao Liu, Adrianto Wirawan, and Bertil Schmidt. CUDASW++ 3.0: accelerating Smith-Waterman protein database search by coupling CPU and GPU SIMD instructions. *BMC Bioinformatics*, 14:117, 2013. doi: 10.1186/1471-2105-14-117.

Svetlin A. Manavski and Giorgio Valle. CUDA compatible GPU cards as efficient hardware accelerators for Smith-Waterman sequence alignment. *BMC Bioinformatics*, 9(Suppl 2):S10, 2008. doi: 10.1186/1471-2105-9-S2-S10.

Torbjørn Rognes. Faster Smith-Waterman database searches with inter-sequence SIMD parallelisation. *BMC Bioinformatics*, 12:221, 2011. doi: 10.1186/1471-2105-12-221.

Tristan Bepler and Bonnie Berger. Learning protein sequence embeddings using information from structure. *International Conference on Learning Representations*, 2019.

Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences.

Proceedings of the National Academy of Sciences, 118(15):e2016239118, 2021. doi: 10.1073/pnas.2016239118.

Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. ProtTrans: toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.